BudgetFlow Project Documentation

About

This document explains the entire structure of the backend and frontend services being used in my BudgetFlow web application. For shorter, and straight to the point introduction, please refer to the README.md in the repository. This application follows a modern three-tier architecture, with a backend service connected to a database, focusing on serving information to the frontend client side. This project is just a passion project, nothing to do with any academic courses.

Frontend

For our Frontend, we use React 19, with TypeScript, as well as Vite as the build tool. React Router allows simple navigations from URL to URL. React Query allows us to easily manage our states. We utilize CSS Frameworks like Tailwind for simple styling, as well as compatibility with ui/component libraries like shadcn/ui and Radix UI. We generate our financial reports and analytics by using a popular JavaScript library, Chart.js, allowing us to create Donut Charts, Pie Charts, Graphs, etc. The goal is to keep the ui/ux as clean and light weight as possible, fully focusing on user comfort, readability, and accessibility.

Backend

For the Backend, we use the popular Express.js backend framework, with TypeScript support. The Backend services handle all HTTP Requests from the client, including GET, POST, PUT, and DELETE, as well as any authentication tokens. (will explain later…) We utilize Express Router for modular and clean RESTful API endpoints, ensuring that we can categorize endpoints in corresponding directories. Some core services include tracking user transactions, and the distance to reaching their budget goals for selected categories, as well as user financial reports, and an integrated AI assistant providing insights on user spending. The AI assistant utilizes DeepSeek-V3 Model, utilizing the OpenAI library.

Database

The database is the core of the backend services, as it allows all data to be specific and secure to a target user. We used Supabase with the free tier to handle user authentication, and for storing user data, such as their transactions, budget targets, goals, etc. We designed a Schema that ensures that each user, when registering, is assigned a UUID, that plays the role of a Primary Key in the database, as well as each user transaction having a Foreign Key referencing a budget target.

Problems Faced

One problem I faced was in the frontend, as I had much less experience developing user interfaces compared to handling server-side logic. For example, I had implemented the frontend so that all the relative components would re-render upon data changes in the backend, this approach was feasible early on, but became a major issue once the interface got more complex. Due to the excessive re-renders, the user interface was laggy and delayed, so I switched over to webhooks, which allowed zero delay updates to the client.

Another problem faced was authentication. The issue was: “Ok, I have confirmed that the user signed in, but how can I confirm that the future requests and calls to our Backend server are from this user? We need something that is sent back that identifies the user, but it also cannot be the UUID (PK) that the user holds, because that’s insecure.” The solution was actually really simple, just use a JWT (Json Web Token), where, on login, the user is assigned a unique and random JWT, that Supabase generates, and it is stored in localStorage, and it is valid all the way until user is done their session.